RELATÓRIO TÉCNICO · 2ª AVALIAÇÃO

NetTopo contra sistemas reais

Correção dos 14 defeitos da primeira avaliação, memória persistente com revalidação, rate limit por equipamento, cofre de credenciais e um laboratório que reprova qualquer divergência do gabarito. Metodologia, resultados, implementação, desempenho e qualidade da versão 0.2.0.

DOCUMENTO	VERSÃO AVALIADA	EXECUÇÃO	COMPARADA COM	DATA	RUNNER
NT-AVL-2026-02 · rev. 2	NetTopo 0.2.0 · 01b1808	run 38427060651 (suíte completa)	v0.1.0 · run 36256925269	13/10/2026	ubuntu-24.04 · 4 vCPU · 16 GB · KVM

PÚBLICO
Engenharia de redes e desenvolvimento

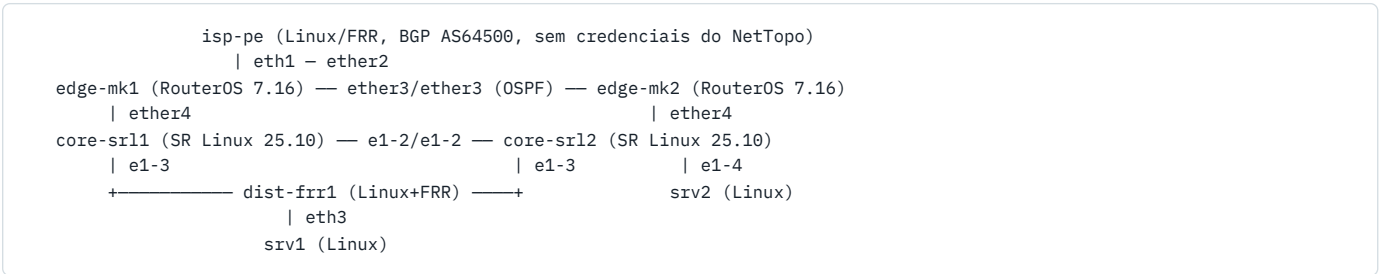
Objetivo e escopo

A primeira avaliação (NT-AVL-2026-02, 26/09/2026) registrou 14 defeitos na versão 0.1.0 (D1 a D14, além do D0, corrigido durante a própria avaliação). Esta revisão avalia a versão 0.2.0 no mesmo laboratório e cobre:

- a correção de cada defeito, com a evidência no laboratório;
- as funções novas pedidas para uso contínuo: **memória persistente com revalidação**, **rate limit por equipamento**, **cofre de credenciais** e **varredura opcional** da rede de gerência;
- um laboratório mais rápido, com **suítes rápida e completa** e **critério automático de aceite**: a rodada é reprovada se qualquer cenário divergir do gabarito.

Ambiente

ITEM	VERSÃO / CONFIGURAÇÃO
Execução	GitHub Actions, `ubuntu-24.04`, 4 vCPU, 16 GB, `/dev/kvm` habilitado
Orquestração	Containerlab 0.79.0
MikroTik	RouterOS 7.16.2 CHR em VM QEMU/KVM (vrnetlab v0.21.0), imagem em cache do Actions
Nokia	SR Linux 25.10.5, JSON-RPC, NETCONF + OpenConfig, LLDP com `hello-timer 5`
Linux	Debian 12, FRR, lldpd (anúncio a cada 5 s) com AgentX, net-snmp, OpenSSH; usuário de monitoração com `sudo -n` restrito a `lldpcli` e `vtysh`
NetTopo	0.2.0, Python 3.12, commit `01b1808`



Gerência em `172.20.20.0/24`, alcançável pelo servidor de descoberta; os endereços dos links de dados não são. A topologia e as credenciais são as mesmas da primeira avaliação.

Metodologia

O gabarito (`lab/ground_truth.yaml`) descreve os 8 equipamentos, os 9 links físicos com as interfaces nativas de cada lado e as 2 adjacências de roteamento, e agora também a **expectativa de cada cenário**: quem deve ser coletado, quem deve aparecer como "não coletado" e com qual motivo, quais links devem existir, o status esperado na memória, os eventos de histórico, os limites de carga e a igualdade com outro cenário.

`lab/scripts/score.py` conta como **erro** qualquer uma destas situações:

- link que não deveria existir, link duplicado, link faltando ou porta errada em qualquer ponta;
- equipamento duplicado, inesperado, coletado quando não deveria ou não coletado quando deveria;
- fabricante ou papel errado;
- adjacência OSPF/BGP esperada não detectada;
- o mesmo equipamento coletado mais de uma vez na mesma descoberta;
- memória que não reaproveitou o esperado, evento de histórico ausente ou inesperado;
- intervalo entre requisições abaixo do perfil de carga ou mais sessões simultâneas que o permitido;
- resultado diferente do cenário de referência (determinismo);
- falha na API, na verificação, nas exportações, nos comandos da CLI ou segredo encontrado em disco.

Com `--strict`, o job termina com erro se a soma for maior que zero. Um equipamento que nenhuma informação disponível permite alcançar não é erro: o gabarito exige que ele apareça como "não coletado" **com o motivo**.

SUÍTE	QUANDO RODA	CENÁRIOS
Rápida	a cada push em `src/`, `lab/` ou no workflow	01 semente única · 02 semente + varredura (cria a memória) · 03 segunda execução com memória · 04 só SSH · 05 só SNMP · 06 APIs · 14 falha + verificação · 15 restauração
Completa	diariamente às 06:00 (Brasília), manualmente ou com `[lab completa]` no commit	a rápida + 07 DNS · 08 credenciais erradas primeiro · 09 escopo · 10 sequencial · 11 SNMPv3 · 12 carga conservadora · 13 determinismo · 16 API, cofre e UI · CLI

Antes e depois

INDICADOR	V0.1.0		V0.2.0	OBSERVAÇÃO
Coletados, semente única (todos os métodos)	4 de 7	→	**6 de 7 · 7 de 7**	sem DNS/memória/varredura · com qualquer um deles
Coletados, inventário de IPs de gerência	5 de 7	→	**7 de 7**	fusão de hosts Linux eliminada (D1)
Só SSH	0 de 7 · 118,1 s	→	**7 de 7 · 16,3 s**	banner, prompt e sondas (D4)
Só SNMP v2c	1 de 7	→	**7 de 7**	com varredura; LLDP-MIB do SR Linux e do lldpd
Precisão de links (melhor configuração)	82%	→	**100%**	100% em todos os 15 cenários de descoberta
Portas corretas	50 a 89%	→	**100%**	endpoints autoritativos (D2)
Links de gerência desenhados	8 de 11 cenários	→	**0**	LLDP em mgmt ignorado (D5)
Nó extra do par BGP	8 de 11 cenários	→	**0**	par junta-se ao nó do link físico (D12)
Rotas RouterOS 7 lidas	1 de 6	→	**6 de 6**	registro terse por linha (D7)
Determinismo	não	→	**sim**	13 idêntico ao 02 (links, portas e coletados)
Requisições por equipamento (todos os métodos)	—	→	**27 a 44**	eram 91 a 98 antes do complemento SNMP enxuto
Revalidação da rede conhecida	—	→	**3,2 s**	descoberta inicial 32,4 s; 0 recoletas
Testes automatizados	46	→	**90**	inclui saídas reais do laboratório como fixtures
mypy	22 erros	→	**0**	
bandit alto	1	→	**0**	hashes não criptográficos marcados
Rodada do laboratório	16 min 19 s	→	**6 min 54 s · 11 min 58 s**	rápida a cada push · completa diária

v0.1.0: run 36256925269-1 · v0.2.0: run 38427060651-1.

Resultados por cenário

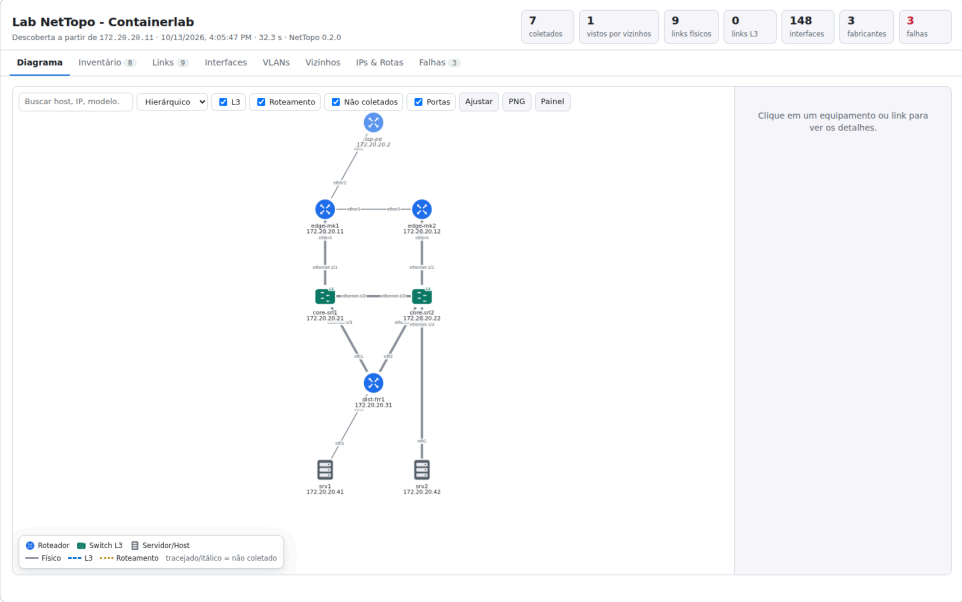
CENÁRIO	ERROS	COLETADOS (ESPERADOS)	LINKS	MEMÓRIA	TEMPO (S)
01 Semente única, sem DNS nem memória	✓ 0	6/6	9/9	—	29,7
02 Semente única + varredura da gerência	✓ 0	7/7	9/9	—	32,4
03 Segunda execução, usando a memória	✓ 0	7/7	9/9	7 confirmados · 0 recoletados	3,2
04 Somente SSH	✓ 0	7/7	9/9	—	16,3
05 Somente SNMP v2c + varredura	✓ 0	7/7	9/9	—	21,5
06 APIs (REST, JSON-RPC, NETCONF)	✓ 0	7/7	9/9	—	16,6
07 Semente única + DNS	✓ 0	7/7	9/9	—	29,3
08 Credenciais erradas primeiro	✓ 0	7/7	9/9	—	19,1
09 Profundidade 1 + exclusão	✓ 0	2/2	5/5	—	14,8
10 Inventário, 1 coleta por vez	✓ 0	7/7	9/9	—	55,9
11 Somente SNMPv3	✓ 0	3/3	4/4	—	8,1
12 Carga conservadora (500 ms, 1 sessão)	✓ 0	7/7	9/9	—	68,6
13 Repetição sem memória (determinismo)	✓ 0	7/7	9/9	—	32,5
14 Verificação após falha injetada	✓ 0	6/6	7/7	4 confirmados · 2 recoletados · 1 inacessível	25,3
15 Verificação após restaurar	✓ 0	7/7	9/9	5 confirmados · 2 recoletados	11,7
16 API, cofre e interface web	✓ 0	descoberta e verificação pela API, cofre, 5 exportações, nenhum segredo em disco			11,1
Comandos da CLI	✓ 0	vendors, discover, verificar, inventario, report e diff com código 0			—

Cada cenário é comparado com a expectativa do gabarito (`lab/ground_truth.yaml`); qualquer erro reprova a rodada.
Fonte: `metrics.json` do run 38427060651-1.

Leitura dos cenários:

- **01 (29,7 s).** Sem DNS, memória nem varredura, `edge-mk2` só é anunciado pelo IP de trânsito `10.255.0.2`. A ferramenta tenta esse IP, depois memória, DNS e o MAC do chassis nas tabelas ARP coletadas, e registra o motivo: "IP anunciado 10.255.0.2: sem resposta ... nenhum outro IP de gerência conhecido (configure DNS, memória/inventário ou varredura)". Os 9 links aparecem, inclusive os do `edge-mk2`, vistos pelos vizinhos.
- **02 (32,4 s).** A varredura de `172.20.20.0/24` (254 endereços, portas e SNMP em paralelo por endereço) encontra o `edge-mk2` e grava a memória. Também acha o gateway do Docker (`172.20.20.1`, SSH sem credencial válida), listado em falhas e fora do desenho, como deve ser.
- **03 (3,2 s).** Segunda execução: 7 checagens leves, 7 confirmações, nenhuma recoleta e nenhum evento.
- **04 (16,3 s).** Só SSH, partindo do inventário: RouterOS identificado pelo banner `ROSSSH`, SR Linux e Linux pelo prompt e por comandos de sonda após login genérico.
- **05 (21,5 s).** Só SNMP v2c com varredura: o SR Linux responde com a LLDP-MIB e o `lldpd` exporta a dele via AgentX. A adjacência OSPF não é cobrada aqui, porque o RouterOS não exporta a OSPF-MIB; a BGP sim.
- **06 (16,6 s).** REST no RouterOS, JSON-RPC no SR Linux e SNMP nos Linux, sem complemento SNMP.
- **07 (29,3 s).** Com DNS, o `edge-mk2` é alcançado pelo nome após o IP anunciado falhar.
- **08 (19,1 s).** Três credenciais erradas antes das certas: mesmo resultado, com a credencial que funcionou por fabricante tentada primeiro nos equipamentos seguintes.
- **09 (14,8 s).** Profundidade 1 e exclusão da rede dos servidores, como configurado.
- **10 (55,9 s).** Sequencial (1 coleta por vez) a partir do inventário.
- **11 (8,1 s).** SNMPv3 authPriv: coleta os três Linux (únicos com usuário v3) e mostra os SR Linux como vizinhos não coletados.

- **12 (68,6 s).** Perfil conservador: menor intervalo medido de 500 ms e 1 sessão em todos os equipamentos.
- **13 (32,5 s).** Repetição do 02 sem memória: coletados, links e portas idênticos.
- **14 (25,3 s) e 15 (11,7 s).** Verificação após congelar o `srv2` (`docker pause`) e derrubar `dist-frr1:eth2` : `srv2` inacessível, `core-srl2` e `dist-frr1` alterados, dois eventos `link_removido` . Depois da restauração: `srv2` com evento `voltou` e os 9 links de volta.
- **16 (11,1 s).** API com credenciais só no cofre (backend arquivo), descoberta por referência, verificação pela API, inventário com 7 ativos, exportações HTTP 200 e nenhum dos segredos do laboratório encontrado no diretório de dados.



Relatório HTML do cenário 02. Cada link mostra a porta de cada lado com o nome nativo; dist-frr1 aparece na camada de distribuição, abaixo do núcleo.

Correção dos defeitos

ID	DEFEITO (V0.1.0)	CORREÇÃO	ONDE	EVIDÊNCIA NA V0.2.0
D1	Hosts Linux fundidos pela "série" `Virtual Machine`	Facts Linux com marcadores `NT_*` ; lista de séries genéricas; série ambígua deixa de identificar; conflito de hostname impede fusão	`vendors/parsers/linux.py` , `discovery/identity.py` , `utils.is_generic_serial`	3 hosts Linux distintos em todos os cenários
D2	Links duplicados (porta descrita de formas diferentes)	Cada ponta usa o nome do próprio equipamento; porta anunciada resolvida por nome, descrição ou MAC; fusão por união de conjuntos	`topology/builder.py` (`resolve_port` , `links_from_halves`)	0 links duplicados; portas 100%
D3	Sem alternativa quando o IP anunciado não responde	Alvo com candidatos: memória, anunciado, DNS, ARP; cache negativo por IP	`discovery/scope.py` , `discovery/engine.py`	`edge-mk2` coletado em 02, 03, 07; motivo explícito em 01
D4	Só-SSH dependia do SNMP/autodetect	Banner sem autenticar, login genérico com prompt e sondas; regex de banner e prompt nos perfis	`discovery/fingerprint.py` , `collectors/ssh.py` , perfis YAML	04: 7 de 7
D5	LLDP da gerência virava link	Interfaces `mgmt*` , `fxp0` , `me0` , `ma*` , OOB ignoradas (regex configurável)	`topology/builder.py` , `config.ignore_interfaces`	0 links mgmt
D6	L3 inferido com IPs iguais	Exige IPs distintos de host, ignora gerência, loopback e interface operacionalmente desligada	`topology/enrich.py`	sem sub-rede falsa; se L3 no link derrubado (:
D7	Rotas RouterOS 7 sem índice descartadas	Saída `terse` : um registro por linha	`vendors/parsers/mikrotik.py`	6 de 6 rotas

ID	DEFEITO (V0.1.0)	CORREÇÃO	ONDE	EVIDÊNCIA NA V0.2.0
D8	SR Linux por CLI sem parser	Parser de tabelas ASCII; LLDP em JSON (sufixo `as json` da CLI); driver JSON-RPC| as json `); driver JSON-RPC |` vendors/parsers/generic.py `,` srlinux.py `,` collectors/rest.py`	LLDP com IP de gerência; 04 e 06 completos	
D9	Vizinho sem IP não seguido	MAC do chassis procurado nas tabelas ARP coletadas; nome via DNS/memória	`discovery/engine.py` (`_late_candidates`)	teste de regressão; 05 completo
D10	OSPF/BGP do RouterOS ausentes	`/routing/ospf/neighbor` e `/routing/bgp/session` (REST e CLI); FRR por `vtysh ... json`	`collectors/rest.py` , `vendors/parsers/mikrotik.py` , `frr.py`	OSPF e BGP em todos cenários aplicáveis
D11	`lldpctl` fora do `PATH`	`lldpcli -f keyvalue` com alternativas `/usr/sbin` e `sudo -n` ; a variante aceita é lembrada por fabricante	perfil `brasil_linux.yaml` , `collectors/ssh.py`	LLDP dos Linux por SS
D12	Par BGP virava nó separado	Par sem resolução junta-se ao nó do outro lado do link físico da sub-rede	`topology/enrich.py` (`routing_adjacencies`)	0 nós extras
D13	Série/modelo ausentes	OpenConfig `components` , `/platform/chassis` (JSON-RPC), licença do CHR	`collectors/netconf.py` , `collectors/rest.py`	SR Linux 7220 IXR-D2 série do CHR
D14	Erro de autenticação mal descrito	Erros agregados por método ("autenticação recusada para N credenciais")	`collectors/orchestrator.py` , `collectors/ssh.py`	motivo correto no `ispe`

Além dos defeitos: Linux só vira roteador com software de roteamento em execução ou rotas aprendidas por OSPF/BGP; a ordem do grafo e os ids de link são determinísticos; um roteador interno ligado ao núcleo e a servidores vai para a camada de distribuição, o que evita que o link `edge-mk1` – `edge-mk2` seja desenhado por cima dele.

Memória persistente e revalidação

O inventário (`inventory/store.py`) guarda, por equipamento: chave estável (hostname curto; na falta dele, o IP de gerência), IPs, perfil, **acesso que funcionou** (método, nome da credencial, credencial SNMP), assinatura da vizinhança (hash dos pares interface local → vizinho, independente do protocolo e do método de coleta), último dado completo, datas e status. Também guarda links, um histórico de eventos (`novo` , `alterado` , `inacessivel` , `voltou` , `link_novo` , `link_removido`) e os IPs que não responderam (cache negativo de 6 h).

Fluxo de `discover_with_memory` (`inventory/revalidate.py`):

1. Para cada equipamento conhecido, em paralelo e respeitando o rate limit: teste da porta do acesso salvo (≤ 3 s), depois **checagem leve** (identidade + só a tabela de vizinhos).
2. Mesma identidade e mesma assinatura → dados reaproveitados, `verificado_em` atualizado. Vizinhança diferente ou dado com mais de 24 h → coleta completa, começando pela credencial que funcionou. Sem resposta → nó "inacessível desde ..." com o motivo.
3. Varredura (se configurada), descoberta normal a partir dos vizinhos (só o que é novo) e gravação.

CENÁRIO	CONHECIDOS	CONFIRMADOS	RECOLETADOS	INACESSÍVEIS	EVENTOS	TEMPO (S)
02 primeira execução	0	0	0	0	7 × `novo`	32,4
03 segunda execução	7	7	0	0	nenhum	3,2
14 após a falha	7	4	2	1 (`srv2`)	`alterado` × 2, `inacessivel` , `link_removido` × 2	25,3
15 após restaurar	7	5	2	0	`alterado` × 2, `voltou`	11,7

No cenário 14, o tempo é dominado pelo `srv2` : um contêiner congelado ainda completa o handshake TCP no kernel, então a sessão SSH espera o banner até o timeout. Um equipamento realmente desligado não responde TCP, e o teste de porta antes da checagem leve encerra a tentativa em até 3 s.

Rate limit

`src/nettopo/ratelimit.py` cria um *throttle* por equipamento (IP, depois ajustado pelo perfil e hostname) com semáforo de sessões, intervalo mínimo entre requisições e contador de falhas de autenticação. Ele vale para cada PDU SNMP, requisição REST, RPC NETCONF, comando SSH, teste de portas e tentativa de login, tanto no código assíncrono quanto nas threads do Netmiko e do ncclient (via `ContextVar`).

PERFIL	INTERVALO	SESSÕES/EQUIP.	SNMP MAX-REP. / WALKS	ATRASO SSH	FALHAS DE LOGIN	PARALELISMO
conservador	500 ms	1	10 / 1	1,0 s	2	4
equilibrado (padrão)	100 ms	1	25 / 2	0,2 s	3	16
rapido	0	2	50 / 4	0	5	64

`overrides` aplicam outro perfil ou valores por CIDR, perfil de fabricante ou regex de hostname. A telemetria (`requisicoes`, `espera_aplicada_s`, `menor_intervalo_ms`, `max_sessoes_simultaneas`, `falhas_autenticacao`) vai para cada equipamento e aparece no painel de detalhes do relatório.

Na primeira rodada da v0.2.0, o perfil equilibrado deixou a coleta de cada equipamento em cerca de 10 s, porque o complemento SNMP percorria de novo tudo o que a API ou o SSH já tinham trazido (91 a 98 requisições). Agora o complemento pede só os grupos que faltam (`covered_groups`): 27 a 44 requisições e cerca de 3,5 s por equipamento, ou seja, menos carga e mais velocidade.

Cofre, API e segurança

- **Cofre** (`vault.py`): backend `keyring` (Windows Credential Manager, Keychain, Secret Service) ou arquivo `vault.enc` cifrado com Fernet e chave derivada por scrypt de `NETTOPO_VAULT_PASSWORD`. O índice (`vault_index.json`) guarda só nome, tipo e usuário. Configurações referenciam `{ref: nome}` ou usam `usar_cofre: true`; `nettopo cred import` move credenciais de um YAML para o cofre.
- **API:** `GET/POST/DELETE /api/credentials` nunca devolvem segredos; `nettopo serve` recusa escutar fora de `127.0.0.1` sem `NETTOPO_API_TOKEN`.
- **Verificado no laboratório (16):** nenhum dos segredos do laboratório aparece em nenhum arquivo do diretório de dados (banco, inventário, índice do cofre).
- **Pendente:** verificação de certificado TLS e de chave de host SSH/NETCONF continua desligada por padrão; o relatório HTML ainda carrega o Cytoscape.js de CDN.

Desempenho

ETAPA DO JOB	V0.1.0	V0.2.0 RÁPIDA	V0.2.0 COMPLETA
Imagem MikroTik CHR	2 min 55 s (build)	13 s (cache)	18 s (cache)
Subir o laboratório (+ instalar o NetTopo em paralelo)	1 min 37 s + 20 s	82 s	72 s
Boot e configuração	52 s	58 s	62 s
Cenários	9 min 08 s (11)	202 s (8)	463 s (17)
Qualidade de código	48 s	no `ci.yml`	51 s
Total do job	**16 min 19 s**	**6 min 54 s**	**11 min 58 s**

Medidas das rodadas 38425932003 (rápida) e 38427060651 (completa). O ganho vem do cache da imagem do CHR, da instalação em paralelo com a subida do laboratório, da qualidade de código fora da suíte rápida e de cenários mais rápidos (complemento SNMP enxuto e LLDP de 5 s, que encurta as esperas da falha injetada).

MEDIDA (V0.2.0)	RESULTADO
Descoberta com varredura do /24 de gerência	32,4 s
Revalidação da rede conhecida (7 equipamentos)	3,2 s
Só SSH, 7 equipamentos	16,3 s
Sequencial (1 coleta por vez)	55,9 s
Perfil conservador (500 ms, 1 sessão, 4 em paralelo)	68,6 s

Matriz fabricante × método

SISTEMA	SNMP V2C	SNMPV3	SSH/CLI	APIS
MikroTik CHR 7.16	✓ funciona	· não se aplica	✓ funciona	✓ funciona
Nokia SR Linux 25.10	✓ funciona	· não se aplica	✓ funciona	✓ funciona
Linux / FRR	✓ funciona	✓ funciona	✓ funciona	· não se aplica

Coleta por método isolado. “Não se aplica”: o método não existe/não foi configurado nesse sistema no laboratório.

Cenários de método isolado: 05 (SNMP v2c), 11 (SNMPv3), 04 (SSH) e 06 (APIs). No laboratório, só os Linux têm usuário SNMPv3 e os Linux não têm API.

Parsing com saídas reais

EQUIPAMENTO	IDENTIFICAÇÃO	HOSTNAME	INTERFACES	IPS	LLDP
`edge-mk1`	✓ funciona 1 reg · mikrotik_facts	✓ funciona 1 reg · mikrotik_identity	✓ funciona 5 reg · mikrotik_interfaces	✓ funciona 4 reg · mikrotik_ip_address	· não se aplica
`core-srl1`	✓ funciona 1 reg · kv_colon	· não se aplica	· não se aplica	✓ funciona 59 reg · generic	✓ funciona 4 reg · srl_json_l
`dist-frr1`	✓ funciona 1 reg · linux_facts	✓ funciona 1 reg · first_line	✓ funciona 4 reg · linux_links	✓ funciona 4 reg · linux_addrs	✓ funciona 3 reg · lldpctl_ke

Saídas reais dos comandos do perfil de cada fabricante, interpretadas pelos parsers do NetTopo (registros extraídos · parser usado). Fonte: `raw/parse-matrix.json`.

As saídas brutas deste laboratório (RouterOS, SR Linux, `lldpcli`, facts Linux) estão versionadas em `tests/fixtures/real/` e alimentam os testes de regressão de D1, D7, D8 e D10.

Qualidade de código

MÉTRICA	V0.1.0	V0.2.0
Testes (`pytest`)	46 aprovados	90 aprovados
Cobertura	71% de 3.875 instruções	74% de 5.624 instruções
Lint (`ruff`)	sem ocorrências	sem ocorrências
Tipagem (`mypy`)	22 erros	0 erros
`bandit`	1 alto, 9 médios, 5 baixos	0 alto, 9 médios, 10 baixos

Os médios restantes do `bandit` são os mesmos da primeira avaliação: comparações com `0.0.0.0` em rotas (B104, falso positivo) e `Markup` sobre o CSS/JS do próprio pacote (B704).

Limitações

- Três famílias de sistema (RouterOS, SR Linux, Linux). Os demais perfis não foram testados com equipamento real.
- Escala pequena: 8 equipamentos e 9 links.
- No laboratório, o SR Linux informa a série `Sim Serial No.` (imagem virtual). A ferramenta a trata como série genérica e não a usa para identificar o equipamento.
- A falha "equipamento fora do ar" é simulada com `docker pause`; um equipamento físico desligado se comporta de forma diferente (sem TCP), caso tratado pelo teste de porta mas não exercitado aqui.

Plano de melhorias

PRIORIDADE	AÇÃO	ESFORÇO (ESTIMATIVA)
1	Laboratórios Huawei VRP, Fortinet e VyOS (Redes Brasil) e Cisco IOS-XE/NX-OS (DevNet) na suíte completa	2 semanas
1	Verificação de TLS e de chave de host ligada por padrão, com exceções explícitas	3 dias
2	Teste de escala com 50 a 100 nós FRR no Actions; medir revalidação e memória	1 semana
2	Cytoscape embutido no HTML (redes isoladas) e layout para redes grandes	1 semana
3	IPv6, VRF e integração com NetBox	3-4 semanas

Reprodução

```
# No GitHub: Actions → "Simulação em laboratório (Containerlab)" → Run workflow (suíte completa)
# ou um push com "[lab completa]" na mensagem do commit
git clone -b lab-results https://github.com/tanatielly/network-discovery-automation resultados
python lab/scripts/score.py resultados/runs/38427060651-1 --strict        # erros em relação ao gabarito
python docs/relatorios/build.py resultados/runs/38427060651-1 --pdf --antes resultados/runs/36256925269-1
```

Arquivos do run: `metrics.json` (erros e métricas por cenário), `summary.md`, `raw/` (saídas brutas e `parse-matrix.json`), `scenarios/<id>/` (topologia, eventos, `timing.json`, logs, relatórios), `memoria/` (inventário persistente dos cenários com memória), `lab-state/`, `quality/` e `logs/`.